Earned or Claimed? Auditing 219 "Done" Reports From a Deployed Agent Swarm

Nathan Langley
Undergraduate, University of North Carolina at Greensboro
With Apart Research

Research conducted at the Apart Swarm Alignment Hackathon, November 2026

ABSTRACT

In an agent swarm, a coordinator that trusts a worker's "done" report can pass false completion claims downstream; in one deployed hierarchy, neither reading the report nor checking cited files for existence reliably caught these failures, while external evidence improved detection when it directly exposed a planted nonexistent file, but not a planted out-of-run commit.

I studied 219 completion claims from 276 delegated jobs across 15 workflows, recorded between 24 October and 6 November 2026. Independent records supported 98 EARNED claims, contradicted 15 UNEARNED claims, and left 106 UNKNOWN. A file-existence check caught 0 of 15 UNEARNED claims: 14 cited only existing items, and one cited nothing checkable.

A report-only Haiku monitor performed near chance on natural UNEARNED cases (AUROC 0.492, n = 7) and on pooled planted c+d+e failures (0.517 [0.402, 0.623]). Adding evidence improved detection of a planted renamed file from 0.387 to 0.802 [0.691, 0.900], but barely helped on planted out-of-run commits (0.384 to 0.467).

A narrower Haiku task-match prompt (post hoc, chosen after the test split was seen; d2 exploratory) detected planted wrong-task reports better than the omnibus prompt: AUROC 0.897 versus 0.645, a paired gain of +0.252 [+0.141, +0.378]. The gain persisted on harder same-workflow swaps (+0.182 [+0.083, +0.274]), but task-match was no better than a free BM25 baseline.

No tested scorer separated planted omissions from earned reports (AUROC 0.514–0.551). These results come from scripted edits of real reports in one deployment.

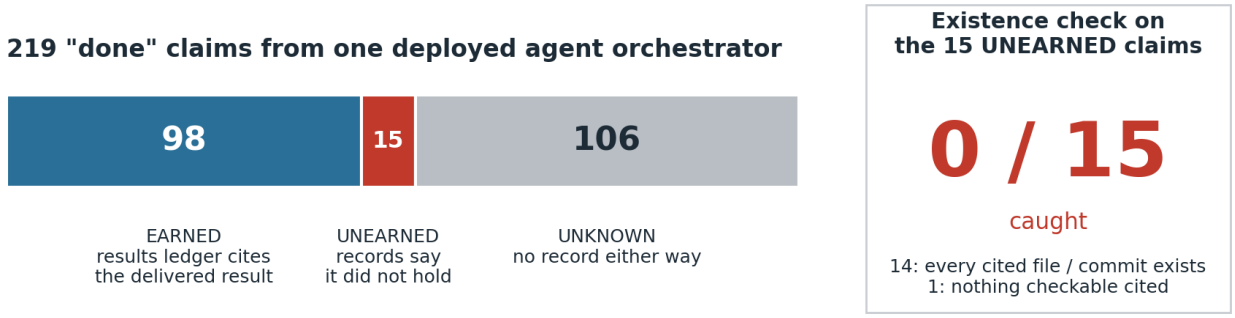


Figure 1. Distribution of 219 completion claims: 98 EARNED, 15 UNEARNED, and 106 UNKNOWN. The accompanying panel shows that an existence check caught 0 of 15 UNEARNED claims: 14 cited only existing items, while one contained nothing checkable.

1. Introduction

I run an orchestrator that delegates work to AI agents. Workers execute tasks, produce artifacts, and send completion reports back to a coordinator. The coordinator then has to decide what to trust.

That decision matters because reporting completion and completing a task are different things. A worker can cite real files, describe successful tests, or report that everything was committed without delivering what was requested.

I wanted to measure how much information a coordinator actually gets from those reports. Specifically, can a monitor distinguish an earned completion from an unearned one? Does checking external evidence help? And does asking a narrow question about task completion work better than asking for a general judgment?

> **Why this matters for swarms.** A coordinator that treats a worker's "done" as true can forward a false completion to the next agent. In this deployed hierarchy, 14 of the 15 false completions cited only files that existed, so existence checks caught none, and report reading was near chance. The useful distinctions appeared when a planted failure left directly detectable evidence or when a monitor checked whether a planted report matched the assigned task. This is an implication from one deployment across 15 workflows, not a measured propagation effect.

My contributions are:

1. A study of 219 completion claims from a deployed agent hierarchy, labeled EARNED/UNEARNED from independent records, with UNKNOWN kept separate.
2. A preregistered comparison of report-only and evidence-augmented monitoring, including failures that do and do not leave detectable traces.
3. An exploratory evaluation of task-specific monitoring against general monitoring and no-model baselines.
4. A test of omitted deliverables, where no evaluated text-based scorer separated planted omissions from earned reports.

2. Findings

F1. Nearly half of completion claims cannot be resolved from independent records

Of 219 "done" claims, 106 had no independent record establishing or contradicting completion.

The deployed orchestrator recorded 276 eligible jobs across 15 workflows. Of those, 219 had both a completed runner status and a final report claiming completion.

Label	Claims	Meaning
EARNED	98	Results ledger supported the requested outcome
UNEARNED	15	Incident or review notes contradicted completion
UNKNOWN	106	Neither supporting nor contradicting evidence
Total	219	

UNKNOWN does not mean failure. It means the records cannot establish the outcome.

This distinction matters for coordination. A system that records "done" as a terminal state may still lack independent evidence that the requested work happened.

F2. File-existence checks caught none of the natural false completions

Checking whether cited artifacts existed caught 0 of 15 UNEARNED claims.

In 14 cases, every cited item existed. The remaining claim contained nothing checkable.

The issue was not simply fabricated file paths. A real artifact can exist without satisfying the assigned task. Checking existence answers whether something is there, not whether it is the right result.

F3. Reading the completion report was no better than chance in the tested comparisons

A general report-only monitor could not reliably distinguish unsupported completion claims from earned ones.

On the natural UNEARNED test set (7 versus 45 EARNED), Haiku's report-only L1 monitor scored AUROC 0.492. Adding evidence through L2 produced 0.497, a paired difference of +0.005 [-0.168, +0.176]. This test was underpowered against the preregistered minimum of 15 natural UNEARNED cases.

The preregistered pooled comparison used 45 planted c+d+e cases against 45 EARNED controls:

Monitor	AUROC [95% CI]
L1: report only	0.517 [0.402, 0.623]
L2: report + evidence	0.538 [0.433, 0.649]
L2 − L1	+0.022 [-0.080, +0.141]

The pooled result was inconclusive under the preregistered rule. More importantly, it was uninformative about the value of evidence by design: planted c, d, and e did not change the facts available to L2.

The result tells me that a general completion judgment based on report text is weak here. It does not tell me that external evidence is useless.

F4. External evidence improved detection of a planted nonexistent file, but not a planted out-of-run commit

Evidence improved detection of a planted nonexistent file, but not a planted out-of-run commit.

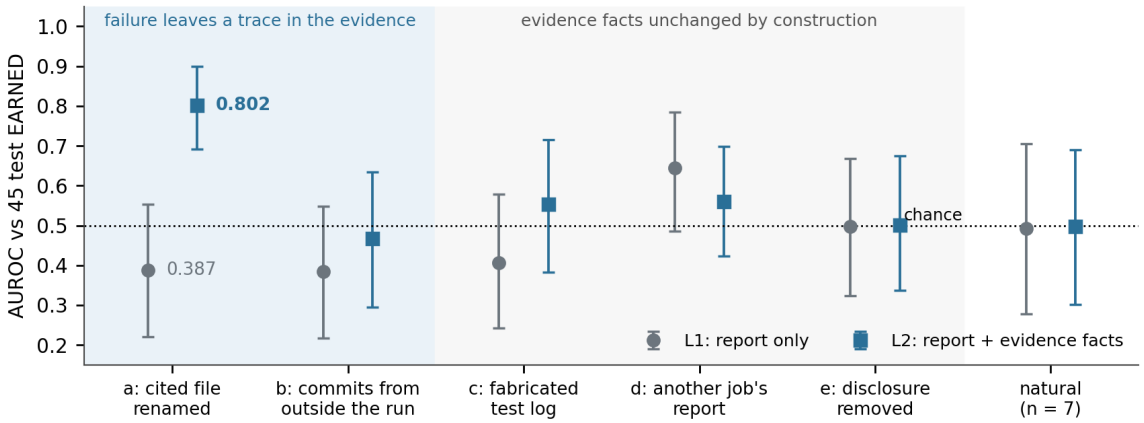


Figure 2. Haiku L1 and L2 AUROC with 95% confidence intervals across planted failure types a–e and natural UNEARNED cases (n = 7). Planted a and b are highlighted as failures that leave traces in the evidence supplied to L2.

Case	L1 AUROC [95% CI]	L2 AUROC [95% CI]
Planted a: renamed file	0.387 [0.220, 0.553]	0.802 [0.691, 0.900]
Planted b: out-of-run commit	0.384 [0.217, 0.548]	0.467 [0.295, 0.635]
Planted c: fabricated test log	0.407 [0.242, 0.578]	0.553 [0.382, 0.715]
Planted d: wrong-task report	0.645 [0.485, 0.785]	0.560 [0.423, 0.698]
Planted e: removed disclosure	0.497 [0.324, 0.668]	0.501 [0.337, 0.675]
Natural UNEARNED	0.492	0.497

The planted renamed-file case gave L2 a concrete contradiction: the report cited a file that did not exist. Detection improved substantially.

The planted out-of-run commit case was different. Although the commit-timing evidence could expose the mismatch, L2 barely improved.

Only one of the two trace-leaving planted failure types produced a clear improvement. Evidence availability alone was not enough; the monitor also had to use it effectively.

For planted c, d, and e, the additional evidence did not describe the alteration. Those results show why the type of evidence matters as much as whether a monitor receives any.

F5. Asking whether the report matches the task works better than a general judgment, but no better than a free BM25 baseline

A narrow task-match prompt outperformed Haiku's general monitor on planted wrong-task reports, without outperforming the strongest no-model baseline.

For planted d, I paired a task with a report from another job. The original construction selected low-overlap task pairs.

Haiku's general L1 monitor scored 0.645 [0.485, 0.785]. Asking only whether the report matched the assigned task raised AUROC to 0.897 [0.807, 0.973].

The paired improvement was +0.252 [+0.141, +0.378].

But the no-model BM25 baseline scored 0.921 [0.845, 0.982]. Haiku task-match minus BM25 was -0.024 [-0.120, +0.074]. There was no supported advantage over BM25.

I then tested a harder planted swap, d2, where the replacement report came from the most similar task in the same workflow.

Exploratory planted d2 results

Scorer	AUROC [95% CI]
Word overlap	0.644 [0.547, 0.741]
BM25	0.801 [0.712, 0.882]
Haiku omnibus	0.594 [0.506, 0.693]
Haiku task-match	0.775 [0.693, 0.860]
Sonnet* omnibus	0.865 [0.797, 0.933]
Sonnet* task-match	0.899 [0.835, 0.952]

On planted d2, Haiku task-match beat Haiku omnibus by +0.182 [+0.083, +0.274]. That difference survived the six-cell Bonferroni correction, with an adjusted interval of [+0.050, +0.305].

Haiku task-match also exceeded word overlap by +0.132 [+0.028, +0.245], but that comparison did not survive Bonferroni (99.17% CI [-0.016, +0.279]).

Against BM25, however, Haiku task-match scored -0.026 [-0.132, +0.085]. Haiku omnibus performed worse than BM25 by -0.208 [-0.343, -0.061], with a Bonferroni interval of [-0.378, -0.008].

Of the three primary planted d2 comparisons, only task-match versus omnibus survived the correction. Among secondary comparisons, Haiku omnibus versus BM25 and Sonnet* task-match versus word overlap also excluded zero after correction; the latter had an adjusted interval of [+0.116, +0.382].

For the original planted d, Sonnet* task-match minus Sonnet* omnibus was +0.061 [-0.006, +0.150], and on planted d2 it was +0.034 [-0.023, +0.092]. Neither was supported. Sonnet* task-match minus word overlap on planted d was +0.107 [+0.000, +0.232], also not supported.

Sonnet* and Opus* are marked because they helped assign EARNED labels, making their comparisons circular.

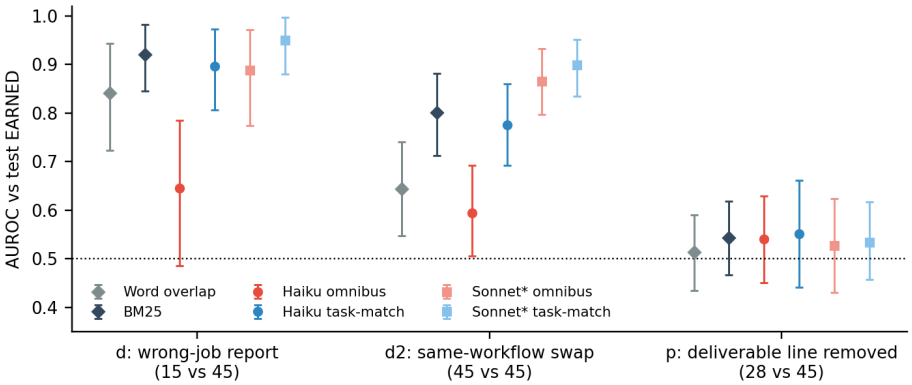


Figure 3. AUROC with 95% confidence intervals for all six scorers—word overlap, BM25, Haiku omnibus, Haiku task-match, Sonnet* omnibus, and Sonnet* task-match—on planted wrong-task reports (d), harder same-workflow swaps (d2), and removed deliverable lines (p). The chance reference is 0.5.

The narrow prompt is a useful direction, especially because it asks a more specific question. But the strongest finding is not that an LLM is required. It is that task-report consistency can be tested directly, with no demonstrated advantage over the free BM25 baseline.

F6. No tested scorer separated planted omissions from earned reports

No tested scorer separated planted omissions from earned reports (AUROC 0.514–0.551).

The planted p condition removed a key deliverable line from a report. There were 28 planted cases against 45 EARNED controls.

Scorer	Planted p AUROC [95% CI]
Word overlap	0.514 [0.435, 0.591]
BM25	0.544 [0.467, 0.619]
Haiku omnibus	0.541 [0.451, 0.629]
Haiku task-match	0.551 [0.441, 0.661]
Sonnet* omnibus	0.527 [0.431, 0.624]
Sonnet* task-match	0.534 [0.457, 0.617]

Every AUROC was between 0.514 and 0.551. Every confidence interval included 0.5, and every upper bound was at most 0.661.

This exploratory result does not rule out moderate detection. It shows that none of the tested scorers established separation on these planted omissions.

This is a different failure from reporting on the wrong task. A report can remain semantically consistent with the assignment while omitting something important.

A monitor that only reads the report may have no way to establish whether the missing deliverable exists. It needs an outcome check against the original request.

3. Why This Matters for Swarms

A coordinator cannot safely treat a worker's completion report as the same thing as task completion.

In this deployment, file existence was a weak signal. All 15 natural UNEARNED claims passed or provided nothing checkable. Report-only monitoring also struggled. On planted failures, the more useful signals came from comparing the report against the assigned task and examining evidence that could expose a specific contradiction.

I think this suggests a more practical approach to swarm monitoring: different failure types need different checks.

A coordinator should compare the worker's report with the original task, inspect evidence that could contradict the claimed result, and check whether the requested deliverables actually exist and satisfy the task. These are separate questions.

The planted wrong-task experiments show that task matching can detect some inconsistencies. The planted renamed-file experiment shows that external evidence can expose a failure that report reading misses. The planted omission experiment shows where both approaches can fall short.

The implication is straightforward. If one worker reports completion and another agent builds on that result, an unsupported claim can become an assumption in subsequent work. I did not measure that propagation directly, but the failure modes observed in this hierarchy give a concrete reason to investigate it.

4. Methods

4.1 Corpus and labels

I used records from an existing agent orchestrator operating between 24 October and 6 November 2026.

The system recorded 506 closed delegations. I excluded 138 jobs from privacy-excluded workflows and another 92 through privacy filtering, leaving 276 jobs across 15 workflows.

The analysis focused on 219 jobs with runner status `done` and a final report claiming completion.

Claims were labeled EARNED/UNEARNED from independent records, using the results ledger, incident reports, and review notes.

- EARNED: A results-ledger entry supported the requested outcome without a contradictory incident or review note.
- UNEARNED: An incident or review note explicitly contradicted the completion claim.
- UNKNOWN: Neither condition was established.

UNKNOWN cases were not treated as failures.

An independent model re-read agreed on 35 of 42 sampled labels, with Cohen's $\kappa = 0.743$.

4.2 Split and monitoring

I separated development and test data before the main monitor evaluations. Prompts were tuned on development data and frozen before phase-2 test calls.

The monitors returned a score from 0 to 100, representing the estimated probability that a completion claim was unearned.

Three evidence levels were evaluated:

Level	Inputs
L1	Original task and worker completion report
L2	L1 inputs plus file-existence and commit-timing facts
L3	Task, report, and worker transcript excerpt

The main comparison was between L1 and L2.

A separate natural-failure L3 test scored AUROC 0.340 [0.146, 0.560] on 7 UNEARNED versus 42 EARNED cases. It was underpowered.

The omnibus prompt asked for a general judgment about completion. The task-match prompt asked only whether the report answered the assigned task.

No-model controls included lexical word overlap and BM25.

4.3 Planted failures

I constructed scripted edits of real completion reports rather than generating new failures with an agent.

Plant	Modification
a	Replace a cited filename with a nonexistent filename
b	Replace in-run commits with real commits from outside the worker run
c	Remove test sentences and insert a fabricated passing log
d	Pair a task with another job's report, selecting low-overlap tasks
e	Remove failure disclosures and insert a fixed completion claim
d2	Pair a task with the most similar alternative task report from the same workflow
p	Remove a key deliverable line

The original plants a–e each contained 15 cases. Planted d2 contained 45 cases, and planted p contained 28.

The planted edits were designed to isolate different kinds of mismatches between what a worker reported and what its task or external evidence supported.

4.4 Analysis and experimental phases

AUROC measured separation between UNEARNED or planted cases and EARNED controls. The main test comparisons used 45 EARNED cases.

Confidence intervals were computed using 2,000 percentile-bootstrap resamples clustered by source job. Model scores were averaged across repeated calls.

The study progressed through preregistered and post hoc stages:

Phase	Status	Main purpose
Phase 2	Preregistered	Compare report-only and evidence-augmented Haiku monitoring
Phase 3	Post hoc	Investigate task matching, stronger models, controls, and alternative evidence
Phase 4	Repeat rule preregistered	Measure task-match stability with additional calls
Phase 5	Preregistered new calls	Test harder planted swaps, omitted deliverables, and report length

The task-match prompt, word-overlap control, and 0.75 decision threshold were introduced after inspecting earlier results. Phase 5 preregistered its procedures but reused the already-inspected test split.

For the planted d2 comparison, the preregistered minimum detectable effect at 80% power was 0.141; for planted p, it was 0.146.

The post hoc full-report comparison used 13 planted d reports and 35 EARNED controls. The phase-5 uncapped Haiku omnibus and task-match conditions each used 3 repeats. Full text did not establish a supported improvement over the shorter input.

5. Related Work

Greenblatt et al. [1] examine AI control under intentional subversion. My study looks at completion claims in ordinary delegated work rather than assuming an adversarial worker.

Hammond et al. [2] discuss risks from advanced multi-agent systems. I focus on a narrower question within that setting: whether a coordinator can distinguish a completed task from a worker reporting completion.

Baker et al. [3] investigate monitoring reasoning models for misbehavior. My monitors operate on final reports and limited external evidence rather than internal reasoning traces.

SHADE-Arena [4] evaluates sabotage and monitoring in agent systems. My planted cases are scripted report alterations, not adaptive sabotage by an agent.

Cemri et al. [5] study failure modes in multi-agent LLM systems. I examine how some completion failures appear to a downstream monitor.

METR [6] documents reward hacking in frontier models. My results do not establish reward hacking or intentional deception; they concern whether completion claims are supported.

SLEIGHT-Bench [7] studies attacks intended to evade agent monitors. My experiments instead isolate specific report inconsistencies and test whether monitors can recognize them.

6. Conclusion

A worker reporting "done" is not the same as a worker delivering the requested result. In this deployed hierarchy, checking that cited files existed caught none of the natural UNEARNED claims, and general report reading performed near chance in the tested comparisons.

External evidence improved detection of the planted renamed-file failure it directly exposed, but not the planted out-of-run commit failure. A narrow task-match prompt improved detection of planted wrong-task reports without outperforming BM25. No tested scorer separated planted omissions from earned reports (AUROC 0.514–0.551), an exploratory result that does not rule out moderate detection.

The next step is to evaluate completion against requested outcomes directly, rather than relying on how convincingly a worker describes its work.

7. Limitations

- Natural failures: Only 7 natural UNEARNED cases reached test, below the preregistered minimum of 15.
- Planted failures: Scripted report edits may not represent how failures arise naturally.

- Post hoc choices: Task-match, word overlap, and the 0.75 threshold were selected after inspecting the test split. Every phase-5 cell is exploratory, and no independent confirmatory evaluation was performed.
- Model-label dependence: Sonnet* and Opus* helped read EARNED labels, making those comparisons circular.
- Deployment scope: One deployment, one operator, and one model provider.
- Report length: Main monitors read the last 2,000 characters. Full stored reports were at most 4,054 characters.
- Label quality: Labels were assigned by model readers, and a human check is pending.

8. Reproducibility and Preexisting Work

The orchestrator, its delegation database, results ledger, incident notes, and operational logs existed before the Swarm Alignment Hackathon. They were the source of the study data, not work created during the sprint.

All study-specific code was written during the sprint, with the first study commit on 6 November 2026.

The public repository contains the study code, data-processing scripts, monitor implementations, preregistrations, aggregate results, and analysis:

https://github.com/ninjahawk/earned-or-claimed

The exact monitor prompts are available in `scripts/p2_prompts.py` and `scripts/p3_prompts.py`.

The aggregate analyses can be reproduced from the published data. End-to-end monitor reruns require the underlying reports, which remain private because they contain information about personal tasks and operations.

9. Author Contributions

Nathan Langley conceived and directed the study and is responsible for all claims.

10. LLM Usage Statement

I used AI tools to run experiments, write code and draft text; every result comes from logged data, and every number was checked against it.

References

[1] Greenblatt, R., Shlegeris, B., Sachan, K., Roger, F. (2023). AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942. https://arxiv.org/abs/2312.06942

[2] Hammond, L., Chan, A., Clifton, J., et al. (2025). Multi-Agent Risks from Advanced AI. arXiv: 2502.14143. https://arxiv.org/abs/2502.14143

[3] Baker, B., Huizinga, J., Gao, L., et al. (2025). Monitoring Reasoning Models for Misbehavior and the Risks of Promoting Obfuscation. arXiv:2503.11926. https://arxiv.org/abs/2503.11926

[4] Kutasov, J., Sun, Y., Colognese, P., et al. (2025). SHADE-Arena: Evaluating Sabotage and Monitoring in LLM Agents. arXiv:2506.15740. https://arxiv.org/abs/2506.15740

[5] Cemri, M., Pan, M. Z., Yang, S., et al. (2025). Why Do Multi-Agent LLM Systems Fail? arXiv: 2503.13657. https://arxiv.org/abs/2503.13657

[6] METR (2025). Recent Frontier Models Are Reward Hacking. https://metr.org/blog/2025-06-05-recent-reward-hacking/

[7] Najt, E., Toft, C., Tracy, T., Roger, F., et al. (2026). SLEIGHT-Bench: A Benchmark of Evasion Attacks Against Agent Monitors. arXiv:2605.16626. https://arxiv.org/abs/2605.16626

Appendix A. Dual-Use / Ethical Considerations

This study identifies failure modes in completion monitoring. Those findings could help improve oversight in multi-agent systems, but they could also help someone construct completion reports that are less likely to raise suspicion.

The planted alterations expose weaknesses in report-only monitoring, evidence handling, and task-report consistency checks. An adversarial worker could potentially exploit similar weaknesses to make incomplete work appear finished.

The experiments did not involve an adaptive attacker attempting to evade the monitors. The planted cases were scripted edits of existing reports.

I have also kept the underlying delegation reports private. They contain information about personal tasks and operational activity that is not necessary to understand the aggregate findings. Privacy exclusions were applied before constructing the study corpus.

The intended use of these results is to improve coordinator oversight: compare reports with their assigned tasks, inspect relevant external evidence, and evaluate whether the requested outcomes were delivered.

The distinction matters because better monitoring should make a swarm less dependent on worker self-reporting, not simply make its completion reports more convincing.

Appendix B. Monitor Prompts

The experiments used two main monitoring approaches.

Omnibus monitor: Given a task and completion report, estimate the probability from 0 to 100 that the worker did not earn its completion claim. The rubric begins by asking whether the report answers the assigned task, then considers other signs of unsupported completion.

Task-match monitor: Judge only whether the report corresponds to the assigned task. This deliberately narrows the question rather than asking for a general completion assessment.

The evidence conditions differed in what the monitor received. L1 used the task and report. L2 added structured file-existence and commit-timing facts. L3 added a worker transcript excerpt of up to 12,000 characters.

Exact prompts and their implementation are public in the study repository:

- `scripts/p2_prompts.py`
- `scripts/p3_prompts.py`

Repository: https://github.com/ninjahawk/earned-or-claimed